Cloud handoff — #495, the stash endpoints share no DTOs

Written: 2026-08-25 · **By:** max (CLI session on Tom's box) · **For:** a cloud Claude Code session on `tom2025b/git-vista`

This is the highest-leverage of the five stash follow-ups, and the least glamorous. Every other read in this app deserialises a type from `git-vista-protocol` on both ends. The stash endpoints alone have every field name written twice, with nothing forcing the two copies to agree.

```
task_id: gv-495-stash-dtos
issue: 495
milestone: M3 — Parallel Work & Recovery [V2]   # a #77 follow-up
repo: tom2025b/git-vista
base: main
branch: refactor/m3.24-stash-dtos
sign_commits_as:
  name: Claude_Max
  email: 262510778+tom2025b@users.noreply.github.com
sign_artifacts_as: max
adr_number: 0079          # ASSIGNED. Do not pick "the next free" c
allowed_paths:
  - crates/git-vista-protocol/src/**
  - crates/git-vista-server/src/handlers/stash.rs
  - crates/git-vista/src/api/stash.rs
  - crates/git-vista/src/features/stash/core.rs
  - docs/adr/
forbidden_paths:
  - design-docs/
  - ci/browser/**            # see "What you cannot run"
  - crates/git-vista/src/features/stash/view.rs
  - crates/git-vista/src/features/stash/signals.rs
  - handoff.md
merge_order: FIRST of all six. See "Merge order". See "Merge order'
```

The gap, precisely

The listing builds its JSON by hand — `handlers/stash.rs`, `stash_list`:

```
serde_json::json!({
    "entry": format!("stash@{{{}}}", s.index),
    "index": s.index,
    "oid": s.oid.0,
    "message": s.message,
    "time": s.time,
})
```

Each write declares its own local struct in that same file: `PushStashRequest`, `StashEntryRequest`, `BranchFromStashRequest`.

And the client declares its own again — `crates/git-vista/src/api/stash.rs` has `PushStashBody`, `StashEntryBody`, `BranchFromStashBody`, and `features/stash/core.rs` has `StashEntry`.

So each field name exists twice in the workspace with no compiler between them.

Why it matters more than it looks

A rename on either side presents as **an empty stash drawer** — not an error, not a 400, just an empty list. That is precisely the failure `git-vista-git`'s `read_stashes` goes out of its way to prevent, and says so:

> An empty `Vec` means the drawer was **read and is empty** … A failure to read returns `Err`; the two are never merged, because "no stashes" and "couldn't look" authorise very different things.

A hand-built JSON object launders exactly that distinction back in one layer up.

And there is now a worked example of the cost. On 2026-08-25 the "Show changes" control was dead on arrival in the shipped drawer: the server's `ShowStashQuery` was `deny_unknown_fields` while the client appends a `?t=` cache-buster to every GET, so every click answered `unknown field \t`, expected `entry` and the drawer rendered that JSON where the patch belonged. Nothing in the Rust suite could see it — the handler was only ever called with a query a test composed by hand, never the one the browser sends. Different mechanism from a rename, same root shape: **the two ends of the wire had no single author.** That fix (commit e270350c`) added three tests pinning the real query string; treat them as the pattern to follow, not as the end of the job.

What "shared DTO" should mean here

Do not just move the structs. The house pattern is a type in `git-vista-protocol` that **both** ends deserialise, with the newtypes that already exist doing the validating:

- `StashSelector` already refuses anything that is not `stash@{<digits>}`. A shared DTO should carry it, not a bare `String` that each end re-checks.
- `CommitOid` exists for oids.
- The listing's `entry` field is *derived* from `index` today (`format!("stash@{{{}}}", s.index)`). Decide whether the wire carries both or one, and say which in the ADR — carrying a derivable field is a second place for it to be wrong, and dropping it moves work to the client.

Add a round-trip test per DTO — serialise from the server's type, deserialise into the client's, assert field-for-field. That is the test that turns a rename from an empty drawer into a compile error or a red test.

What you cannot run, and what to do instead

The browser leg does not run in a cloud container. The server refuses to start without its strict sandbox tier and the kernel there reports `landlock_abi=-1`; INV-13 gives no degraded mode. Installing `bwrap` changes nothing — it is the kernel's capability that is missing.

This task should not need it. But it changes the wire, so **the browser suite is exactly what would catch a mistake here** — which means: when you are done, say in the PR body, explicitly, that `ci/browser/run.sh` has not been run and must be before merge. A session on Tom's box will run it. Do not leave that implicit.

`cargo test --workspace` is yours and must be green. Two tests in `git-vista-server` flake under parallel execution because they race on the process-global current repository (#438): `recovery_center::tests::a_stale_claimed_undo_is_refused_and_the_branch_is_left_alone` and `state::tests::selection_flow_carries_mode_and_gates_writes`. Re-run before believing either.

Acceptance

1. Every stash field name has **one** author at the wire boundary. `grep -n 'expected_oid\|keep_index\|include_untracked' crates/git-vista-server/src/handlers/stash.rs crates/git-vista/src/api/stash.rs` finds only the shared DTO's uses, no second struct definition. **Do not grep these names workspace-wide**: `GitOperation`'s `PushStash`/`PopStash`/`DropStash`/`BranchFromStash` variants already use identical field names for the internal execution plan

in `crates/git-vista-protocol/src/plan.rs` (a separate, already-correct layer), and that name is threaded through `planner.rs`, `planner/stash.rs`, `sandbox/dispatch.rs`, and `git-vista-mcp/plan_tools.rs` — none of which are in `allowed_paths`. Those are legitimate consumers, not the duplicates this task removes. `"entry"` alone is too generic a token to grep meaningfully at all.
2. `stash_list` no longer builds JSON with `serde_json::json!`.
3. A round-trip test per DTO, and each proved able to go red **two different ways** — rename a field, and change a type. One `caught` verdict is not proof.
4. **ADR 0079** records the shape chosen (especially the `entry`-vs-`index` question) and why. `docs/adr/README.md` index updated.
5. `cargo fmt --all`, `cargo clippy --all-targets -- -D warnings`, `cargo test --workspace` green.
6. PR body says `Closes #495`, and states that the browser leg is unrun. **Never delete the branch.**

Merge order

Land this **first** of the stash follow-ups. It rewrites field names across the server handler and the protocol crate; every other stash PR would pay for its rebase, and a mechanical rename is the worst diff to review through a conflict resolution. CLOUD-2 (#493/#494) has been told to wait for you.

Signed: max · 2026-08-25T10:08:00-04:00